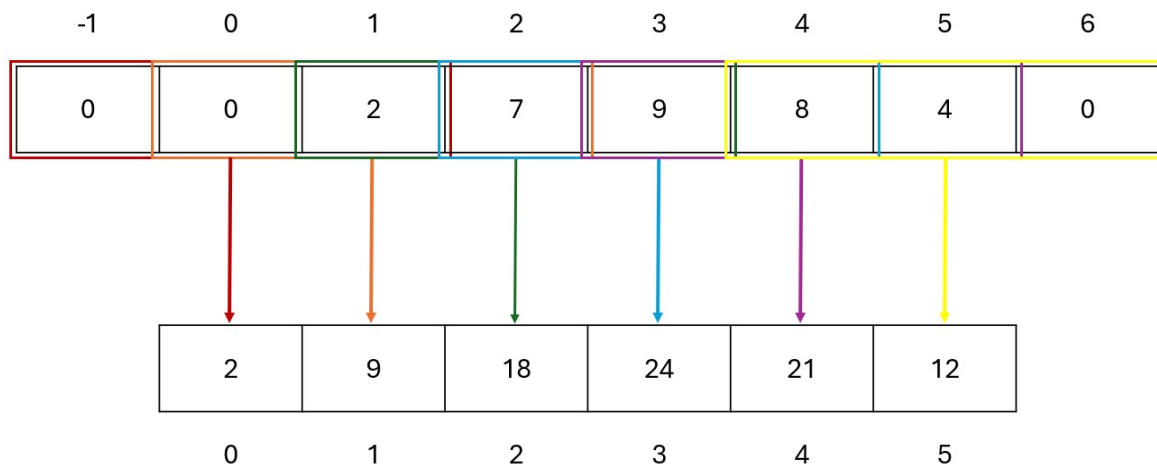


If input is of length n
 kernel is of length k
 then output is of length $n - k + 1$.

If we want output to be the same length as input then we can pad input with $\frac{k-1}{2}$ elements on each end.

Example



Valid padding - only use valid elements from our input (i.e. no padding)

Same padding - pad input such that element has same size as input.

✓ has same size as input.

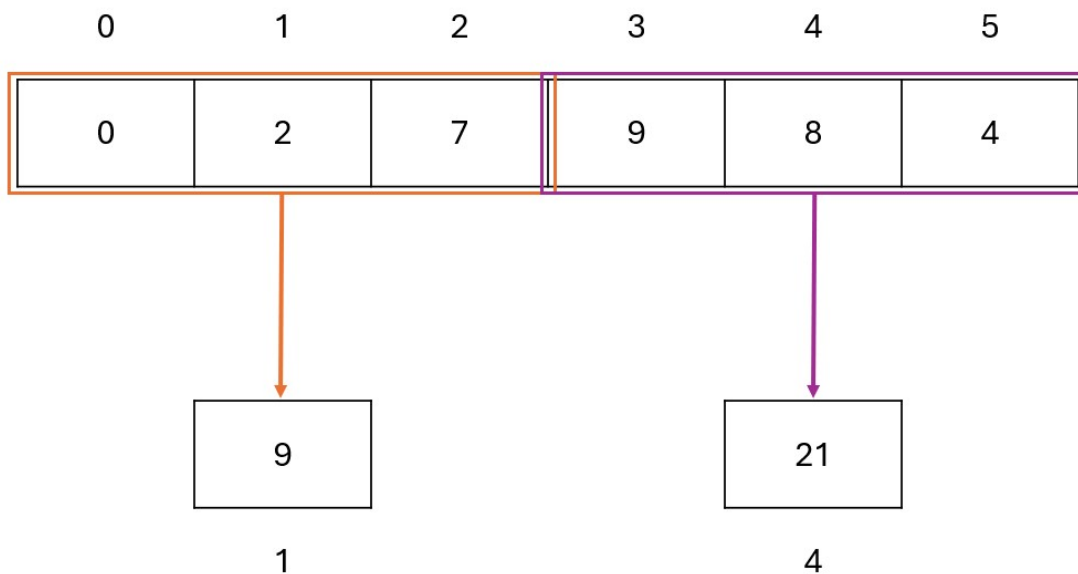
Stride

Only compute outputs $(x * w)(t)$ when

$$\left(t + \frac{k-1}{2}\right) \% s = 0$$

s - stride

Example Kernel: $w(t) = \begin{cases} 1 & \text{if } -1 \leq t \leq 1 \\ 0 & \text{otherwise} \end{cases}$



Stride $s = 3$

Dilation

Skipping some elements in the kernel.

Skipping some
 l - dilation rate

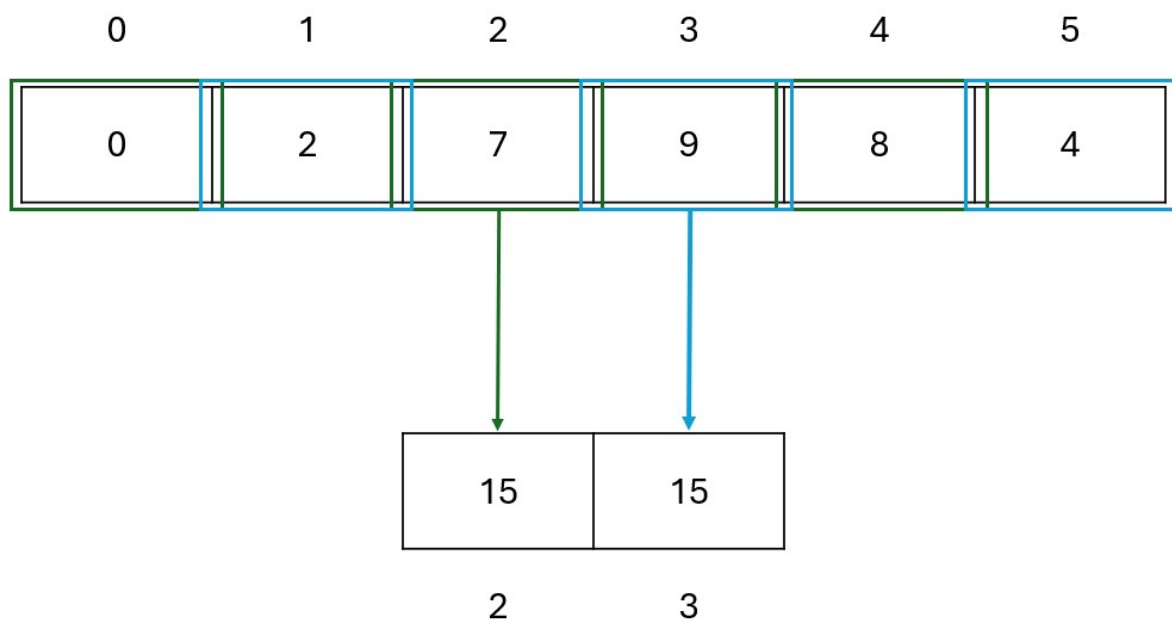
$$(x * w)(t) = \sum_{-\infty}^{\infty} x(a) w\left(\frac{t-a}{l}\right)$$

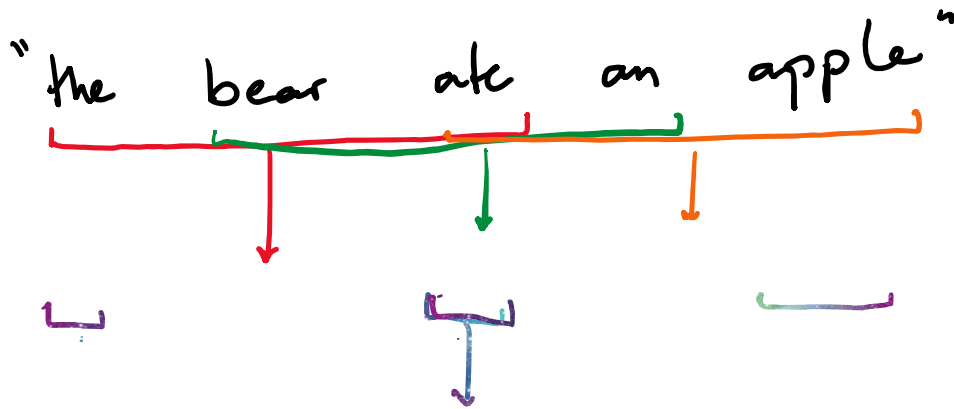
$$w(t) = \begin{cases} \text{non-zero} & \text{if } -c \leq t \leq c \text{ and } t \in \mathbb{Z} \\ 0 & \text{otherwise} \end{cases}$$

Example

Kernel: $w(t) = \begin{cases} 1 & \text{if } -1 \leq t \leq 1 \text{ and } t \in \mathbb{Z} \\ 0 & \text{otherwise} \end{cases}$

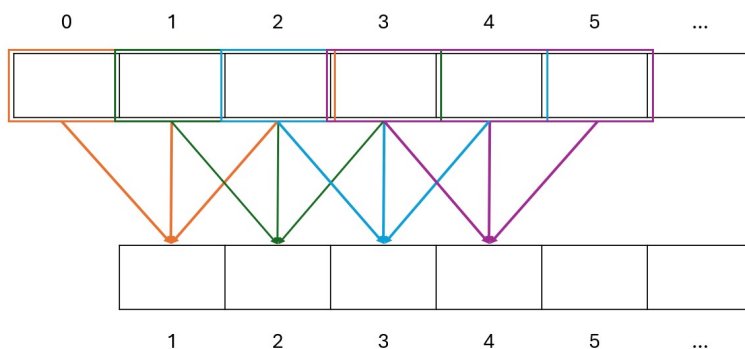
Convolution with dilation 2. Stride $s=1$





Convolution in Neural Networks

For a given output y_i , the set of inputs that affect it are called the receptive field.



Like a fully connected layer with constraints:

1. All weights are zero outside receptive field.
2. Weights for all outputs are the same, but shifted in space.

... .. recurrent layers have?

bu.

What advantages do convolutional layers have?

1. More computationally efficient for modelling short-term dependencies.

Fully connected layer: $(m+1) \times n$ parameters

Convolutional layer: k parameters

m - input length, n - output length, k - kernel width

2. Shared parameters across time/space.
Features learned independent of location.

Pooling

Computes a summary statistic of nearby inputs.

$$y_i = f(x_i, \dots, x_{i+c-1})$$

c - constant (width of pooling operation).

Use with stride (often stride c).

Useful for downsampling.

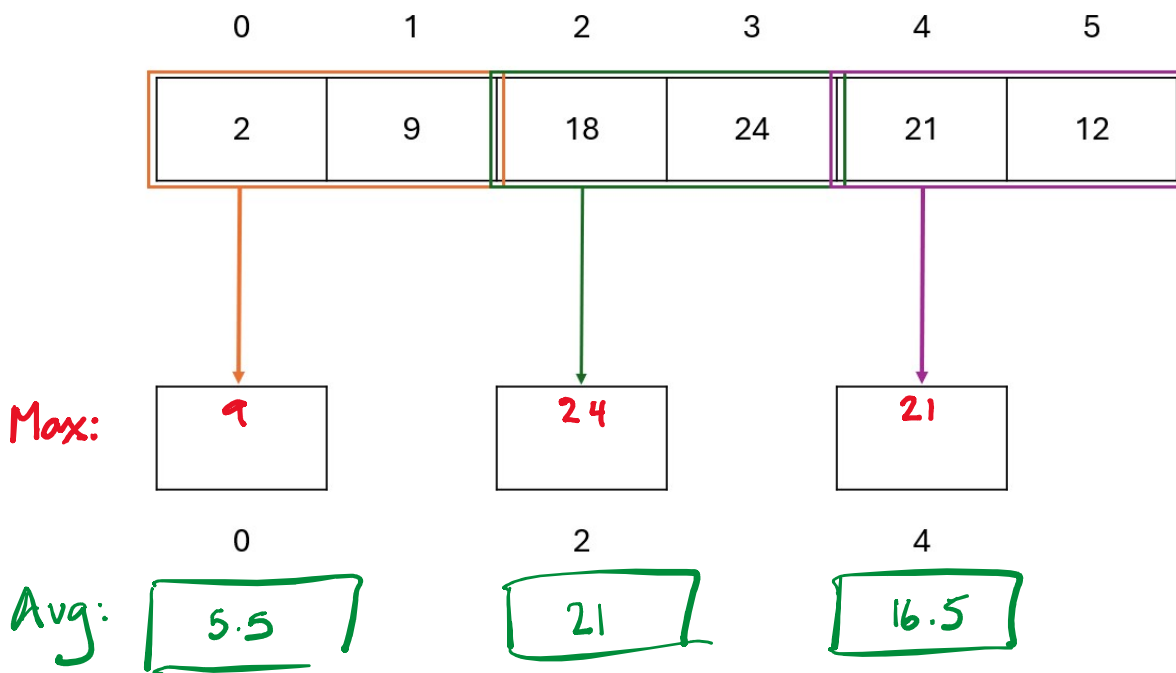
1. Improves efficiency.
2. Makes network invariant to small translations.

Forms of pooling:

1. Max pooling: $f(x_i, \dots, x_{i+c-1}) = \max(x_i, \dots, x_{i+c-1})$
2. Avg pooling: $f(x_i, \dots, x_{i+c-1}) = \frac{1}{c} (x_i + \dots + x_{i+c-1})$

Example

Pooling with width 2, stride 2.



Often pooling is after convolution.

Convolution in Higher Dimensions

Convolution in 2D:

$$(x * w)(s, t) = \sum_{a=-\infty}^{\infty} \sum_{b=-\infty}^{\infty} x(a, b) w(s-a, t-b)$$

Alternatively:

$$(x * w)(s, t) = \sum_{a=-\infty}^{\infty} \sum_{b=-\infty}^{\infty} x(s+a, t+b) w(a, b)$$

Typically define kernel

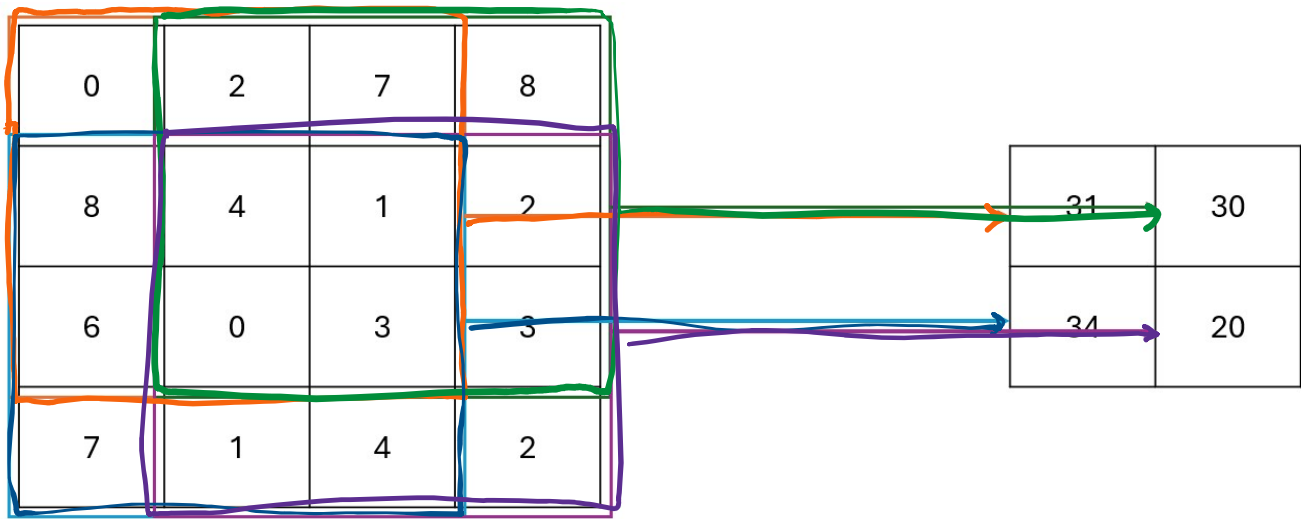
$$w(s, t) = \begin{cases} \text{non-zero} & \text{if } -c \leq s, t \leq c \\ 0 & \text{otherwise} \end{cases}$$

Width - number of non-zero elements in each dimension.

Example

$$w(s, t) = \begin{cases} 1 & \text{if } -1 \leq s, t \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

Kernel is 3×3 & width 3"



Multi - Channel Convolution

$$(x * w)(s, t) = \sum_{a=-\infty}^{\infty} \sum_{b=-\infty}^{\infty} \sum_c x(s+a, t+b, c) w(a, b, c)$$